Vad är DevOps?

Session 1 · Från commit till produktion

Mjukvaruutvecklingsprocessen — DevOps

Arcada · 8.9–28.10.2026

Innan vi börjar: en fråga

Tänk på ert senaste programmeringsprojekt (kurs, hobby, jobb):

Hur kom koden till "användarna"?
Hur lång tid tog det från sista ändringen tills någon annan kunde se den?

Så här ser det oftast ut: `scp`/FTP:a filer till en server, dra en zip till en kompis, eller köra lokalt och bara visa skärmen.

Agenda idag

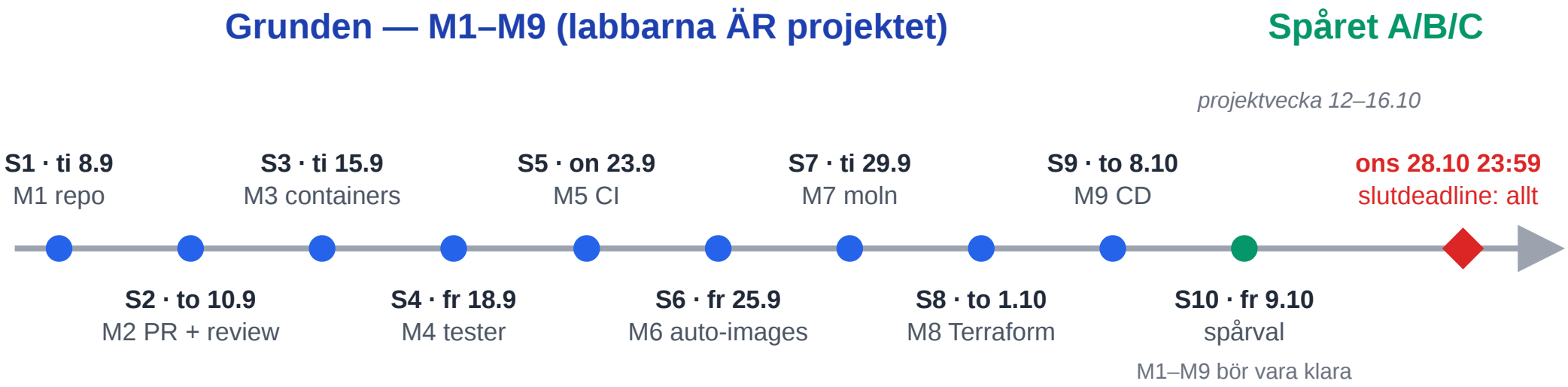
1. Välkommen + öppningsfråga
2. Kursen: upplägg, milstolpar, betyg
3. **Vad är DevOps?** Kultur, CALMS, DORA, livscykeln
4. Paus
5. **Lab M1** — konton, eget repo, första commit
6. Wrap-up: verifiera M1, vanliga problem, nästa gång

Kursen i en mening

> Ni bygger **en tjänst — från första commit till automatisk produktion** —
> och varje veckas lab lägger en permanent milstolpe på samma repo.

- **Labbarna ÄR projektet.** Kursen består av labbar och en större slutuppgift — det ni gör idag ligger kvar i samma repo den 20 oktober.
- **Manuellt före automatiserat.** Ni gör allt för hand först — så att automatiseringen känns som en befrielse, inte magi.
- Appen får ni färdig (frontend + backend). Kursen handlar om **processen**, inte om att koda appen.
- Ni kommer möta mycket konfiguration — målet är aldrig att memorera den, utan att förstå vad varje fil gör och var man letar.

Kursresan: 9 sessioner, 9 milstolpar



Varje milstolpe avslutas med en **tagg** i repot och har ett tydligt **bevis**.

Arbete som inte syns i repot dokumenteras kort i `inlamning/`.

Betyg (max 109 p): M1–M9 är hela grunden, bonusen ger höjden

Del	Poäng
Milstolpar M1–M9	0–11 p per milstolpe (helhetsbedömning) = max **99**
Bonus: repo-hygien, commit-historik, PR-disciplin, fungerande kedja	**+10**
Max	**109**

Betygsgränser: 0–50 Underkänt · 51–60 = 1 · 61–70 = 2 · 71–80 = 3 · 81–90 = 4 · 91+ = 5

Tumregel: en fullt genomförd milstolpe ≈ ett betygssteg (t.o.m. M6 = 66 p = 2, t.o.m. M8 = 88 p = 4, alla nio = 99 p = 5).

Hårda krav utöver poängen

- Godkänt kräver **minst 51 p OCH individuell reflektion** (~1 sida, `inlamning/REFLEKTION.md`)
- Hård slutdeadline för allt (M1–M9 med milstolpsbevis + reflektionen): **ons 28.10 23:59** — repot bedöms från to 29.10

Alla nio milstolpar klara + reflektionen inlämnad = klar med kursen.

Bonus +10 p — vad "kvalitet" betyder

- **Repo-hygien** — rimlig `.gitignore`, inga secrets committade, tydlig struktur
- **Commit-historik** — många små commits med tydliga meddelanden, inte en jättecommit i slutet
- **PR-disciplin** — featurebranches, PR med review innan merge till main
- **Fungerande kedja** — varje milstolpe har verifierbart bevis

Par är välkomna (rekommenderas!) — men betyget är alltid **individuellt**: Git-historiken visar vem som gjort vad, och milstolpsbevisen i `inlamning/` visar **var och en** sin del av kedjan, med namn.

Praktiskt

- **När:** 10 lektionstillfällen 8.9–9.10 · 165 min per pass · hård slutdeadline för allt ons 28.10 23:59 (inget demopass — repot bedöms)
- **Var:** schema enligt Arcadas system — tider varierar (ti/to/fr/må)!
- **Par eller solo:** par rekommenderas, bildas idag i labben
- **Material:** kursens material-repo + itslearning; appen kommer från ett template-repo på GitHub
- **Verktyg:** allt är gratis — GitHub (Education), CSC:s moln (via Haka)
- **Kontocheck:** GitHub, GitHub Education (valfritt tips), CSC via Haka — vi gör det tillsammans som steg 0 i dagens lab, ingen förberedelse hemma behövs

Del 2: Vad är DevOps?

DevOps är inte...

- ...ett **verktyg** man köper (”vi har Jenkins, alltså har vi DevOps”)
- ...en **titel** (”vi anställde en DevOps så nu är det löst”)
- ...ett **team** som sitter mellan Dev och Ops *(varning: en ny silo!)*

DevOps är...

> En **kultur och ett arbetssätt** där de som bygger och de som driftar delar ansvar för hela kedjan — så att kod går från commit till produktion **snabbt, säkert och upprepbart**.

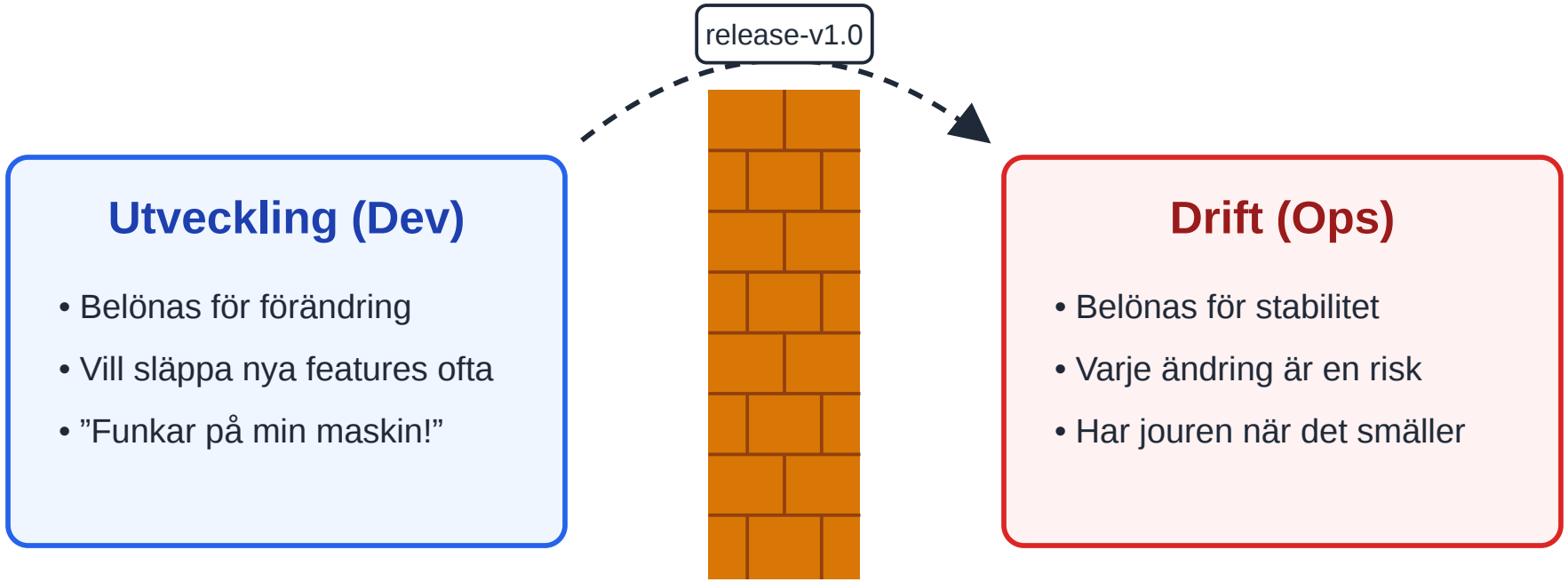
Ni har redan gjort ”ops” — för hand

I skolprojektet var ni både dev och ops: någon kopierade filer till servern kl 02, och bara den personen visste hur.

Det håller för tre personer och en tjänst. Lägg till folk, tjänster och riktiga användare — så spricker det på ett av två sätt:

- **Alla gör allt** — kaos, ingen vet vad som körs
- **Vi specialiserar** — Dev här, Ops där… och en mur emellan

Muren



”the wall of confusion”

Var har ni sett en sådan här mur? (skola, jobb, hobbyprojekt...)

CALMS — fem linser på DevOps

		I den här kursen
C	Culture — delat ansvar, blame-fri felsökning	par + review, avslutande reflektion
A	Automation — bygg, test, deploy utan handpåläggning	CI/CD (S5–S6, S9), IaC (S8)
L	Lean — små steg, snabb feedback, kort kö	små PR:ar, täta releaser
M	Measurement — mät det som spelar roll	DORA går att räkna ut på ert repo
S	Sharing — kunskap och verktyg delas	demos, gemensam template

CALMS — det finstilta

Ärlig varudeklaration:

- **Kultur går inte att köpa.** Verktygen är den lätta delen — beteenden (vem vågar säga ”jag gjorde sönder det”?) är den svåra.
- **Automatiserar du en trasig process får du en snabb trasig process.**
- **L är den bortglömda bokstaven** — stora releaser *känns* säkrare men är farligare.
- **Mätvärden som blir mål slutar mäta** (Goodharts lag) — gör deployment frequency till mål och folk deployar tomma commits.

DORA: fyra nyckeltal

Forskningsprogrammet DORA (*Accelerate*, Forsgren m.fl.) — fyra mått som tillsammans fångar både **fart** och **stabilitet**:

Fart	Stabilitet
Deployment frequency — hur ofta når kod produktion?	**Change failure rate** — hur stor andel av ändringarna orsakar fel i produktion?
Lead time for changes — commit → produktion, hur länge?	**Failed deployment recovery time** — hur snabbt är ni tillbaka efter ett haveri?

I slutet av kursen **kan ni** räkna ut dessa fyra på ert eget repo.

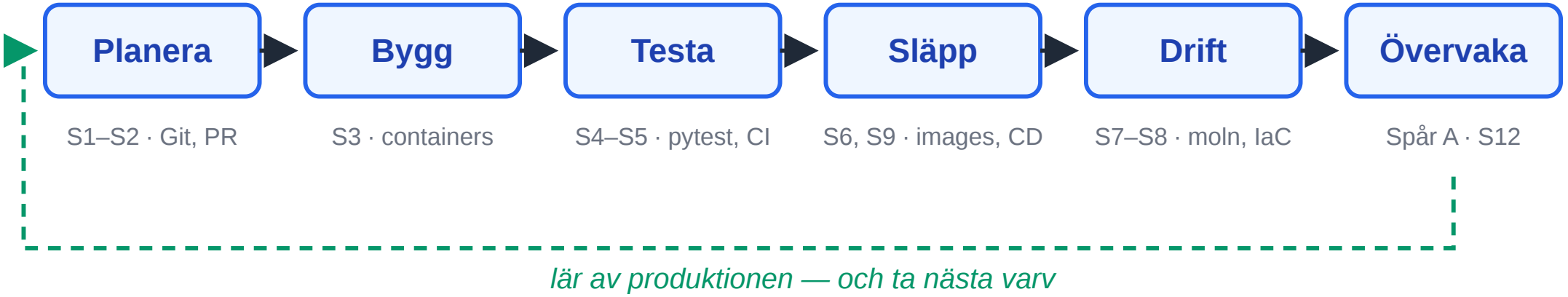
Var ligger ribban? (DORA 2024)

Nivå	Deployfrekvens	Lead time	Change failure rate	Recovery
Elite (19 %)	on demand	< 1 dag	5 %	< 1 h
High (22 %)	dag–vecka	1 dag–1 vecka	20 %	< 1 dag
Medium (35 %)	vecka–månad	1 vecka–1 månad	10 %	1 dag–1 vecka
Low (25 %)	månad–halvår	1–6 månader	40 %	1 vecka–1 månad

Källa: DORA *State of DevOps Report* **2024** — sista rapporten med nivåband.

Gissa: var hamnar ett typiskt studentprojekt? Var är **ni** den 20 oktober?

Servicelivscykeln



En tjänst är inte **klar** när koden är skriven — den ska släppas, driftas, övervakas och förbättras. Kursen är **ett varv genom hela cykeln**, på er egen tjänst.

Hållbarhet, på riktigt (SDG 9)

- **FN:s SDG 9 — Hållbar industri, innovation och infrastruktur:** bygga motståndskraftig infrastruktur och uppgradera den till att bli hållbar och resurseffektiv.
- **DevOps roll:** automatisering och IaC gör infrastruktur **pålitlig och reproducerbar**, CI/CD minskar spill — färre trasiga releaser, mindre manuellt omarbete — och `terraform destroy` är en dygd: riv det du inte använder. Molnresurser är riktiga pengar och riktig energi.

Lab: M1

Konton · eget repo · första commit

Vad är en tagg — och varför taggar vi?

En **tagg** är ett namn på en bestämd commit. Till skillnad från en branch **flyttar den sig aldrig** — den pekar på exakt samma kod för alltid.

I riktig DevOps märker taggar releaser (`v1.2.3`): release-pipelinen triggas av taggen, taggen svarar på frågan "vilken kod ligger just nu i produktion?", och den är punkten man rullar tillbaka till när något går sönder.

I kursen är det samma mekanism med ett annat syfte: `m1-repo`, `m2-review` … visar vilka commits som hör till vilken milstolpe. Git-historiken är ert betygsunderlag — taggen är kvittot som gör det mekaniskt kontrollerbart.

Glöm inte: `git push` skickar inte taggar — kör `git push origin m1-repo` separat. Tagga sist (taggen fastnar på HEAD just då): committa → pusha → kontrollera på GitHub → tagga → pusha taggen.

M1 — dagens milstolpe

Mål: ett eget repo, skapat från kursens template, med er första commit — pushad och taggad.

1. **Steg 0** — kontokoll: GitHub, GitHub Education (valfritt), CSC via Haka
2. Skapa ert repo från **template-repot** (”Use this template”)
3. Öppna miljön: **Codespace** (rekommenderas) eller klona lokalt
4. Gör en riktig ändring (t.ex. README.md), committa, pusha
5. Kontrollera committen på GitHub — tagga då: `git tag m1-repo` och pusha taggen
6. Bjud in läraren som collaborator (`Tobias-Eriksson`) och lämna in länken på itslearning

Bevis: repot + commit (användarnamn) + taggen + collaboratorn + länken.

Wrap-up: är M1 klar?

Kör checklistan tillsammans:

- [ ] Repot finns på GitHub — skapat **från template** (inte fork, inte tomt)
- [ ] `git log` visar er commit — med **rätt författarnamn**
- [ ] Committen syns på GitHub (pushen gick fram)
- [ ] Taggen `m1-repo` syns under **Tags** på GitHub
- [ ] I par: **båda** är collaborators, båda har committat
- [ ] Läraren är inbjuden som collaborator, och GitHub-länken är inlämnad på itslearning (**"Milstolpe 1"**)

Fastnade du? Ta problemet **olöst** till session 2 — vi löser det där.

Nästa gång

Session 2 · to 10.9 kl 09:15 — Versionshantering

Branches, merge-konflikter, pull requests, code review.
M2: arbetsflöde med branch protection — par granskar varandras PR:ar, solo granskar sin egen i PR:en.

Ta med: ett fungerande M1-repo (det ni byggde idag).